

Parallel Skeletons: A Detailed Overview

Parallel skeletons are higher-order functions or patterns that encapsulate common parallel computation structures. Think of them as "templates" for parallelism—you provide the sequential logic (the "what"), and the skeleton handles the orchestrations, synchronization, and data movement (the "how").

1. Data Parallel Skeletons

These skeletons operate on large datasets where the same operation is applied to multiple data elements.



Map Skeleton

Concept: Applies a function f to every element in a collection $\{x_1, x_2, \dots, x_n\}$ to produce a new collection $\{f(x_1), f(x_2), \dots, f(x_n)\}$.

- **Parallelism:** Each $f(x_i)$ can be computed independently.
- **Example:** Squaring every number in a list. If you have 4 cores, each core can square 1/4th of the list.



Reduce Skeleton

Concept: Combines all elements of a collection into a single value using a binary operator $\oplus$ (e.g., $+$, $\times$, $\min$, $\max$).

- **Parallelism:** Requires the operator to be **associative** $(a \oplus b) \oplus c = a \oplus (b \oplus c)$. This allows for tree-based reduction (summing pairs in parallel).
- **Example:** Finding the total sum of an array.



Scan Skeleton (Prefix Sum)

Concept: Computes partial reductions. For an input $[x_1, x_2, x_3]$, it produces $[x_1, x_1 \oplus x_2, x_1 \oplus x_2 \oplus x_3]$.

- **Parallelism:** Harder than Reduce because of dependencies, but can be solved in $O(\log n)$ steps using algorithms like Hillis-Steele or Blelloch scan.
- **Example:** Ranking elements or finding offsets in a memory buffer.

2. Task Parallel Skeletons

These focus on distributing different tasks (which might be the same or different functions) across threads.



Farm (Master-Worker)

Concept: A Master node generates tasks and places them in a queue; Workers pick tasks from the queue as they become free.

- **Strength:** Excellent for **Load Balancing**. If some tasks take longer than others, workers who finish early just grab the next task.
- **Example:** A web server pool handling incoming requests.

Pipe (Pipeline)

Concept: Similar to an assembly line. Data flows through a series of stages $\{S_1, S_2, \dots, S_k\}$. Each stage runs in parallel on different data items.

- **Efficiency:** Throughput increases, though individual item latency remains the same.
- **Example:** Video encoding (Read -> Transform -> Compress -> Write).

Divide and Conquer (D&C)

Concept: A recursive pattern. If a problem is small, solve it; otherwise, split it into sub-problems, solve them in parallel, and combine the results.

- **Parallelism:** Creates a tree of tasks.
- **Example:** Parallel Merge Sort.

Wavefront

Concept: Processes elements on a grid where an element (i, j) depends on its neighbors (e.g., $(i-1, j)$ and $(i, j-1)$).

- **Parallelism:** Computation proceeds in "waves" along the diagonals. All elements on the same diagonal can be computed in parallel.
- **Example:** Dynamic Programming (edit distance, LCS) or heat diffusion simulations.